

Capítulo 4

Capa de Red

Algoritmos de enrutamiento en WAN de enrutadores interconectados no jerárquica

Algoritmos de enrutamiento en WAN de enrutadores interconectados no jerárquica

- **Intención pedagógica:** La clase busca que los estudiantes descubran, analicen y deriven progresivamente los algoritmos de enrutamiento utilizados en redes de enrutadores interconectados no jerárquicas.
 - El recorrido comienza con soluciones simples que no se preocupan de los cambios en la topología o el tráfico, y avanza hacia mecanismos capaces de adaptarse dinámicamente a estas variaciones.
- La **meta formativa** es que los estudiantes desarrollen la capacidad de razonar como diseñadores de protocolos de red, no solo como usuarios de algoritmos ya establecidos.
 - Para esto serán confrontado con diferentes situaciones problemáticas que los obligarán a proponer, evaluar y mejorar algoritmos de manera incremental.
 - Al finalizar, deberían ser capaces de explicar por qué un protocolo funciona, qué supuestos hace y cómo responde ante cambios en la red.

Algoritmos de enrutamiento en WAN de enrutadores interconectados no jerárquica

- **Conocimientos previos asumidos:** La clase asume que los estudiantes ya manejan conceptos fundamentales de redes y grafos, entre ellos:
 - Qué es una WAN de enrutadores interconectados no jerárquica y cómo se organiza.
 - Qué es un enrutador y cómo opera una tabla de reenvío.
 - ¿Qué son grafos con pesos en los arcos?
 - ¿Cómo opera el algoritmo de Dijkstra de caminos más cortos?
- Estos conocimientos permiten que la clase se concentre en el diseño de algoritmos distribuidos, sin detenerse en fundamentos ya conocidos.

Algoritmos de enrutamiento en WAN de enrutadores interconectados no jerárquica

- En esta clase vamos a explorar uno de los problemas centrales de las redes de computadoras:
 - cómo lograr que los paquetes encuentren su camino en una red de enrutadores interconectados, incluso cuando la topología cambia, los enlaces fallan o el tráfico se vuelve impredecible.
- Aunque hoy damos por sentado que Internet “simplemente funciona”, detrás de esa aparente simplicidad hay algoritmos distribuidos que deben tomar decisiones correctas con información parcial, incompleta o desactualizada.
 - Nuestro objetivo será reconstruir esos algoritmos desde cero, enfrentándonos a los mismos desafíos que motivaron su creación.
- Empezaremos con soluciones intuitivas y poco sofisticadas, y avanzaremos paso a paso hacia mecanismos robustos capaces de adaptarse dinámicamente a los cambios de la red.

Algoritmos de enrutamiento en WAN de enrutadores interconectados no jerárquica

- A lo largo de esta clase vamos a recorrer progresivamente distintos escenarios de enrutamiento, ordenados según el tipo de desafío que plantean.
 - Comenzaremos con el caso más simple: una red cuya topología no cambia y cuyo tráfico varía muy poco, de modo que las tablas de enrutamiento pueden **calcularse una sola vez y permanecer estables**.
 - Luego avanzaremos hacia un entorno más adverso, donde enlaces o nodos pueden fallar por condiciones hostiles, lo que nos obligará a estudiar **mecanismos de inundación** orientados a **maximizar la probabilidad de entrega**.
 - A continuación analizaremos el caso en que la red no busca llegar a un destino único, sino **difundir información a todos los nodos**, lo que requiere una **variante distinta de inundación**.
 - Finalmente, abordaremos el escenario más general y realista: una red donde la topología y el tráfico pueden cambiar dinámicamente, no por hostilidad sino por variaciones normales de operación, y donde será necesario derivar un **protocolo de estado de enlace** capaz de adaptarse a esos cambios de manera eficiente.
- Este recorrido nos permitirá entender cómo cada algoritmo surge como respuesta natural a un tipo específico de problema.

Bloque 1: Problema del enrutamiento en WANs

- Supongamos que inicializamos las tablas de reenvío en una WAN. **¿Qué puede pasar si no se usan algoritmos de enrutamiento?** (Ver algunas maneras en que se puede degradar el funcionamiento de la red)

Bloque 1: Problema del enrutamiento en WANs

- Supongamos que inicializamos las tablas de reenvío en una WAN. **¿Qué puede pasar si no se usan algoritmos de enrutamiento?** (Ver algunas maneras en que se puede degradar el funcionamiento de la red)
 - Algunos enrutadores pueden quedar inactivos.
 - Los caminos pueden ser innecesariamente largos.
 - Se pueden sobrecargar algunas de las líneas de comunicación y los enrutadores asociados a ellas.

Bloque 1: Problema del enrutamiento en WANs

- **Repaso:** ¿Qué es un algoritmo de enrutamiento?

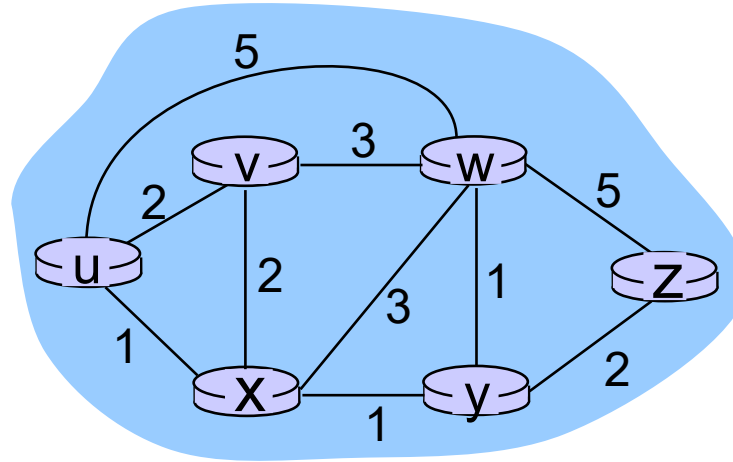
Bloque 1: Problema del enrutamiento en WANs

- **Repaso: ¿Qué es un algoritmo de enrutamiento?**
 - Es un algoritmo que escoge bien las rutas a usarse para enviar paquetes.
 - Las rutas se determinan actualizando y llenando las tablas de reenvío.
 - Un algoritmo de enrutamiento se ejecuta en los enrutadores de la WAN.

Bloque 2: Conceptos básicos

- Antes de comenzar con algoritmos de enrutamiento en redes de enrutadores interconectados vemos algunos conceptos.
- **¿Cómo representar una subred como un grafo?**

Bloque 2: Conceptos básicos



Grafo: $G = (N, E)$

N = conjunto de enrutadores = $\{ u, v, w, x, y, z \}$

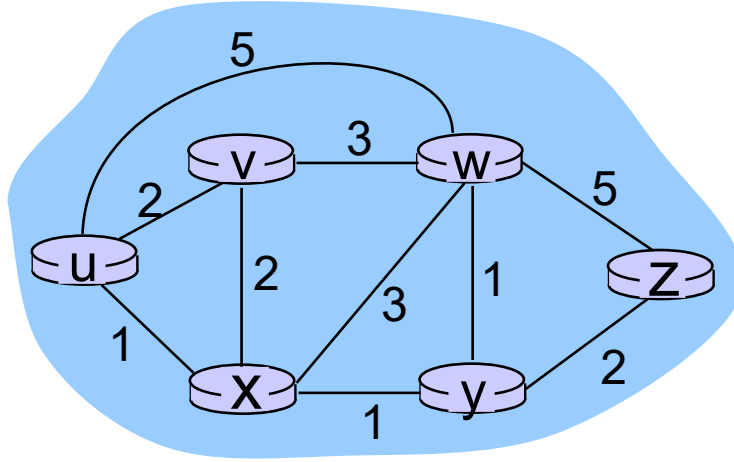
E = conjunto de enlaces = $\{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

Los arcos tienen **etiquetas** para el **costo** de atravesarlos

Repaso:

- ¿Cómo calcular el costo de los arcos?
- ¿Cómo calcular el costo de un camino?

Bloque 2: Conceptos básicos



$c(x,x')$ = costo de enlace (x,x')
e.g., $c(w,z) = 5$

Los costos de los arcos podrían calcularse como función de varios parámetros:

- La distancia, ancho de banda, tráfico medio, costo de comunicación, longitud media de las colas, retardo medio, y otros factores.

Cálculo del costo de un camino:

$$\text{costo del camino } (x_1, x_2, x_3, \dots, x_p) = c(x_1, x_2) + c(x_2, x_3) + \dots + c(x_{p-1}, x_p)$$

Bloque 3: Enrutamiento de caminos más cortos

Introducción

- ❖ En este bloque ingresamos al primer escenario estable: una red cuya topología no cambia y cuyo tráfico varía muy poco.
 - En este contexto, podemos permitirnos calcular las rutas una sola vez y usarlas durante largos períodos sin necesidad de actualizarlas.
 - El objetivo es comprender cómo un enrutador puede determinar, de manera autónoma, el mejor camino hacia cada destino utilizando únicamente la representación de la red como un grafo con costos en los enlaces.
 - Aquí aparece el algoritmo de Dijkstra como herramienta central: un método sistemático para construir el árbol de caminos más cortos desde un nodo hacia todos los demás.
- ❖ Este bloque sienta las bases conceptuales para todo lo que sigue, mostrando cómo se construyen las tablas de reenvío cuando la red es estable y predecible.

Bloque 3: Enrutamiento de caminos más cortos

- **Algoritmo de enrutamiento de caminos más cortos:**
 - ❑ Para elegir una ruta entre un par de enrutadores, encontrar en el grafo una de **las rutas más cortas** entre ellos.
- Algoritmos de cálculo de la ruta más corta entre dos nodos.
 - ❑ Uno de ellos es el algoritmo de **Dijkstra** (1959).
 - Dado grafo conexo con costos en los enlaces, y nodo n en el grafo, obtiene **árbol de caminos más cortos** desde n hacia todos los demás nodos.
 - El árbol de caminos más cortos se representa con un **mapeo** donde para cada nodo del grafo de la subred asigna su padre (en el árbol de caminos más cortos).
 - Repasar los detalles del algoritmo de Dijkstra visto en algoritmos 2.

Bloque 3: Enrutamiento de caminos más cortos

- ❖ **Sugerir los pasos para calcular las tablas de reenvío usando el algoritmo de Dijkstra** (indicar los pasos que vienen antes y después de usar ese algoritmo)

Bloque 3: Enrutamiento de caminos más cortos

- ❖ **Sugerir los pasos para calcular las tablas de reenvío usando el algoritmo de Dijkstra** (indicar los pasos que vienen antes y después de usar ese algoritmo)
 1. Construir grafo de la subred con costos.
 2. Ingresar grafo de la subred con costos en los enrutadores.
 3. En cada enrutador construir tabla de enrutamiento; para eso:
 - a. Ejecutar algoritmo de Dijkstra en el enrutador
 - b. A partir de árbol de caminos más cortos con raíz en el enrutador obtenido generar la tabla de reenvío del enrutador.

Bloque 3: Enrutamiento de caminos más cortos

- ❖ **De caminos óptimos hacia redes impredecibles:** Hasta ahora trabajamos con una red estable, donde los costos no cambian y podemos calcular rutas óptimas con Dijkstra y mantenerlas sin modificaciones.
 - Pero ¿qué ocurre cuando la red deja de ser estable? ¿Qué pasa si un enlace crítico cae justo después de haber calculado la ruta más corta?
 - En el próximo bloque nos moveremos a un entorno hostil, donde la topología puede cambiar mientras un paquete está en tránsito.
 - Allí veremos que los caminos más cortos ya no alcanzan y necesitaremos un mecanismo más robusto: la inundación hacia un destino.

Bloque 4: Inundación hacia un destino

- ❖ En una **red sometida a fallas frecuentes** —por bombardeos, catástrofes naturales o infraestructura extremadamente frágil— queremos mandar un paquete de un nodo de origen u a un destino v .
 - Los algoritmos de caminos más cortos **dejan de ser confiables** porque suponen una **topología relativamente estable**.
 - El cálculo de la ruta óptima se vuelve obsoleto en cuestión de segundos: un enlace o enrutador crítico puede caer justo después de haber sido elegido como parte del camino, dejando al paquete atrapado o descartado en una región vulnerable.

Bloque 4: Inundación hacia un destino

- La **meta** aquí es **maximizar la probabilidad de entrega** de un paquete a un destino específico, incluso cuando la topología cambia cuando el paquete está en tránsito.
- Para ello exploraremos la idea de inundación hacia un destino, un mecanismo que replica el paquete por múltiples caminos simultáneamente.
- Los estudiantes descubrirán tanto su potencia como sus problemas, y derivarán mejoras que permiten controlar duplicados, bucles y crecimiento explosivo del tráfico.

Bloque 4: Inundación hacia un destino

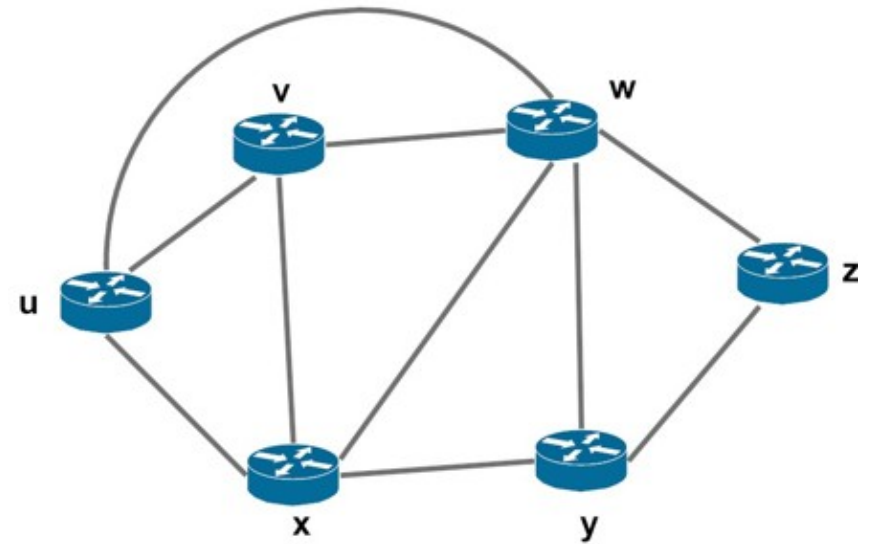
- **Para la situación de red sometida a fallas frecuentes proponer la idea de un algoritmo** (asumir que el grafo tiene varios caminos distintos para llegar a un destino)

Bloque 4: Inundación hacia un destino

- **Para la situación de red sometida a fallas frecuentes proponer la idea de un algoritmo** (asumir que el grafo tiene varios caminos distintos para llegar a un destino)
- para enviar un paquete de un origen u a un destino v se respetan las siguientes reglas:
 - u manda el mensaje por todas las líneas de salida.
 - Cada paquete que llega a un enrutador distinto de v se reenvía por cada una de las líneas excepto aquella por la que llegó.
- A este algoritmo se le llama **inundación**.

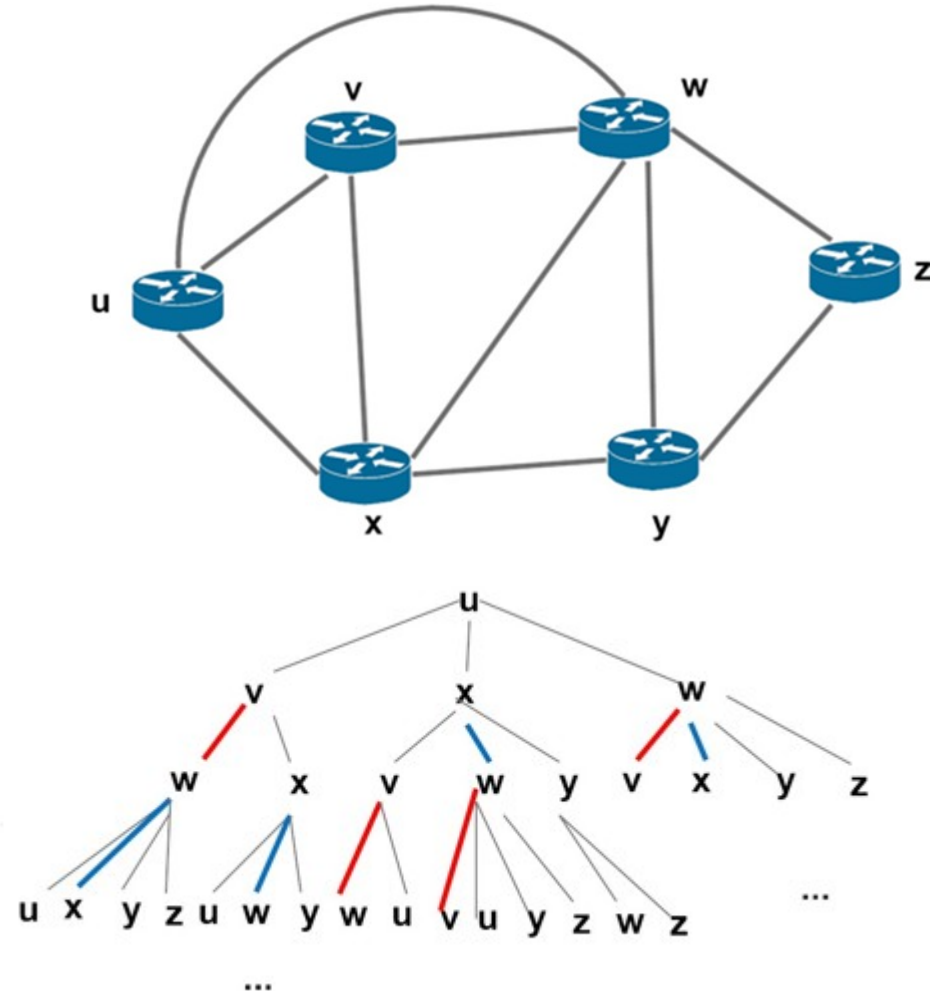
Bloque 4: Inundación hacia un destino

- **¿Qué problemas tiene la idea anterior?**
(considerar por ejemplo la red de la figura)



Bloque 4: Inundación hacia un destino

- **Problemas de la idea anterior:**
 - La inundación genera grandes cantidades de *paquetes duplicados*.
 - Árbol de envío de paquetes. Cada arco representa un paquete que se envía.
 - P.ej. arcos rojos y azules en el árbol de arriba corresponden a paquetes duplicados.
 - Árbol de envío de paquetes es infinito con infinitos duplicados. O sea, se generan infinitas rutas.
 - La causa es la presencia de ciclos en el grafo de la subred.



Bloque 4: Inundación hacia un destino

- Hace falta **limitar** un poco el proceso de inundación dado en la idea anterior para resolver el problema.
- **Proponer ideas de solución al problema** (Pensar que cuando llega un paquete a un enrutador, este chequea una condición sobre el paquete para decidir si lo sigue reenviando).

Bloque 4: Inundación hacia un destino

- Hace falta **limitar** un poco el proceso de inundación dado en la idea anterior para resolver el problema.
- **Proponer ideas de solución al problema** (Pensar que cuando llega un paquete a un enrutador, este chequea una condición sobre el paquete para decidir si lo sigue reenviando).
 - Cada enrutador recuerda los **paquetes difundidos**; por lo tanto si un paquete llega, se chequea si el enrutador lo difundió antes. Si no lo hizo, se lo difunde.
 - El paquete lleva un contador con la **cantidad de saltos que recorrió**. Si ese contador supera un cierto valor (por ejemplo el diámetro de la red) el paquete no se difunde.
 - El paquete lleva un **contador de cuantos saltos le quedan** que se decrementa con cada salto del paquete. Si el contador es cero, entonces el paquete no se difunde.

Bloque 4: Inundación hacia un destino

- Vamos a derivar en detalle el algoritmo de la solución de registro de paquetes difundidos.
- **¿Qué información deben llevar los paquetes que se difunden?** (tener en cuenta que en un enrutador puede usar el mecanismo de inundación para enviar varios paquetes distintos hacia diversos destinos)

Bloque 4: Inundación hacia un destino

- Vamos a derivar en detalle el algoritmo de la solución de registro de paquetes difundidos.
- **¿Qué información deben llevar los paquetes que se difunden?**
- El enrutador de origen pone un **número de secuencia** en cada paquete que recibe de sus hosts (así se distingue entre paquetes distintos del mismo enrutador de origen).
- **¿Qué información debe recordar un enrutador en su registro de paquetes difundidos?** (recordar que un enrutador puede recibir varios paquetes distintos de distintos enrutadores de origen)

Bloque 4: Inundación hacia un destino

- Vamos a derivar en detalle el algoritmo de la solución de registro de paquetes difundidos.
- **¿Qué información deben llevar los paquetes que se difunden?**
- El enrutador de origen pone un **número de secuencia** en cada paquete que recibe de sus hosts (así se distingue entre paquetes distintos del mismo enrutador de origen).
- **¿Qué información debe recordar un enrutador en su registro de paquetes difundidos?**
- Un enrutador recuerda para cada enrutador de origen los números de secuencia recibidos – i.e. pares <enrutador de origen, n°secuencia>

Bloque 4: Inundación hacia un destino

- **¿Qué pasa cuando llega un paquete a un enrutador?** (recordar la información en el paquete y en el registro de paquetes difundidos)

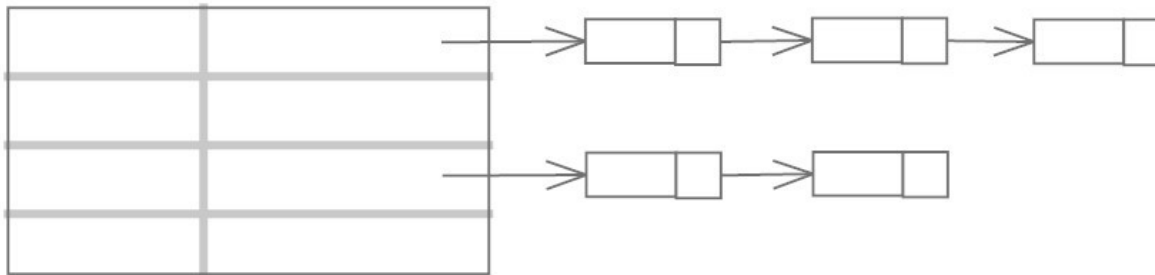
Bloque 4: Inundación hacia un destino

- **¿Qué pasa cuando llega un paquete a un enrutador?**
- Si llega un paquete a un enrutador con par <enrutador de origen, número de secuencia>, verifica si el par está en el registro de paquetes difundidos.
 - Si no está, se lo difunde al paquete.
 - En caso contrario, se lo descarta.
- **Diseñar una estructura de datos para el registro de paquetes difundidos** (¿Cómo representar los paquetes recibidos de un enrutador de origen').

Bloque 4: Inundación hacia un destino

- **Estructura de datos:** Para cada enrutador usar **tabla de registro de paquetes difundidos**.

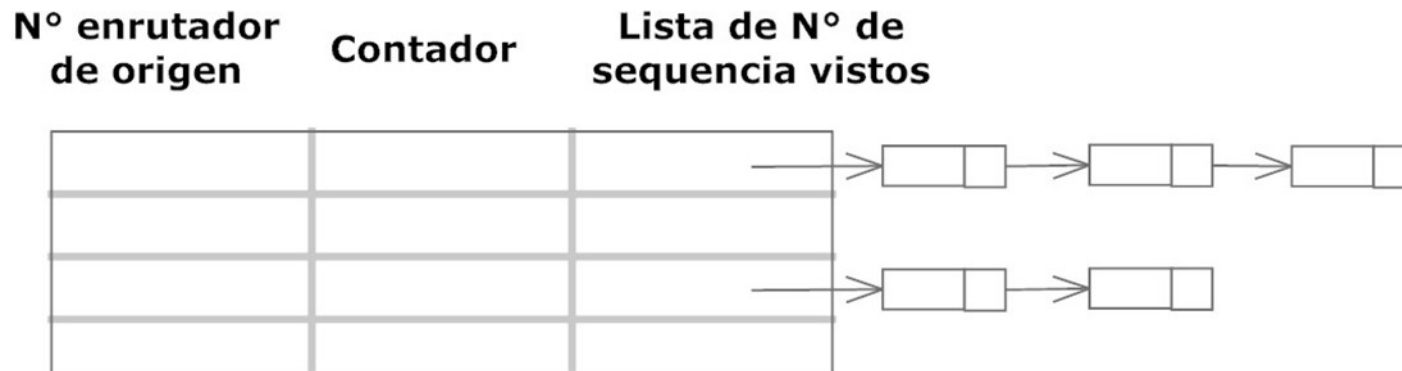
Nº enrutador Lista de Nº de
de origen secuencia vistos



- Esta tabla tiene el problema que las listas enlazadas pueden crecer sin límites. **¿Cómo evitar que esto suceda?** (asumir que manejamos listas ordenadas por número de secuencia y que puede haber lagunas en esas listas)

Bloque 4: Inundación hacia un destino

- ¿Cómo se puede evitar que las listas enlazadas *crezcan sin límites*?
 - ❑ Agregar una columna llamada **contador** que indica el mayor número de secuencia tal que:
 - ❑ Llegaron paquetes con todos los números de secuencia anteriores desde ese enrutador de origen.



Bloque 4: Inundación hacia un destino

De evitar duplicados a controlar el alcance de la inundación

- Con el mecanismo de registro de paquetes difundidos logramos resolver un problema central de la inundación: evitar que los paquetes circulen indefinidamente y se generen duplicados en ciclos.
 - Cada enrutador ahora puede reconocer si ya procesó un paquete y descartar copias repetidas.
 - Sin embargo, este mecanismo no controla hasta dónde se propaga un paquete.
 - En redes grandes, un mensaje puede recorrer toda la subred incluso cuando solo nos interesa explorar una región cercana al origen o cuando el destino está relativamente cerca.
 - Esto genera tráfico innecesario.
- Para limitar el alcance de la inundación y hacerla más eficiente, introduciremos un nuevo mecanismo: el contador de saltos.
 - El mismo reducirá la propagación a un radio controlado y evitando que la inundación se extienda más allá de lo necesario.

Bloque 4: Inundación hacia un destino

- Ahora vamos a deducir el algoritmo de inundación que usa un contador de saltos.
- Asumir que el contador de saltos comienza con un valor inicial que va decreciendo a medida que el paquete se va moviendo. **¿Cómo se determina el valor inicial del contador de saltos?**

Bloque 4: Inundación hacia un destino

- Ahora vamos a deducir el algoritmo de inundación que usa un contador de saltos.
- Asumir que el contador de saltos comienza con un valor inicial que va decreciendo a medida que el paquete se va moviendo. **¿Cómo se determina el valor inicial del contador de saltos?**
 - ❑ Lo ideal es inicializar el contador de saltos a la **longitud de la ruta entre el origen y el destino**.
 - ❑ Si el emisor desconoce el tamaño de la ruta, puede inicializar el contador al peor caso, es decir, al **diámetro total de la subred**.
- **Describir el algoritmo de inundación que usa este contador de saltos.**

Bloque 4: Inundación hacia un destino

- Al comienzo en el enrutador de origen se **inicializa** el contador de saltos del paquete y se reenvía el paquete por todos los enlaces.
- Cuando llega un paquete a un enrutador se **decrementa** el contador de saltos.
- Si el enrutador es el **enrutador de destino**: el paquete no se difunde más.
- Sino si el **contador de saltos es cero**: el paquete se descarta.
- sino el paquete se difunde.

Bloque 4: Inundación hacia un destino

- **Recapitulación:** En este bloque analizamos qué sucede cuando la red opera en un entorno hostil, donde enlaces y nodos pueden fallar sin previo aviso.
 - Descubrimos que los caminos más cortos dejan de ser confiables y que necesitamos un mecanismo que aumente la probabilidad de entrega: la inundación hacia un destino.
 - Vimos su idea básica, sus problemas (duplicados, bucles, crecimiento explosivo) y derivamos mejoras como el registro de paquetes difundidos y los contadores de saltos.
 - Este bloque nos mostró que, cuando la red es inestable, la redundancia es una herramienta esencial

Bloque 4: Inundación hacia un destino

- **De llegar a un destino a llegar a todos:** En el bloque anterior aprendimos a usar la inundación para aumentar la probabilidad de entrega en redes inestables.
 - Pero ese mecanismo estaba orientado a un destino específico. ¿Qué pasa si el objetivo no es llegar a un nodo, sino difundir un mensaje a toda la red?
 - En ese caso, la lógica cambia: ya no buscamos un camino que sobreviva, sino garantizar cobertura total.
 - En el próximo bloque modificaremos la idea de inundación previa y derivaremos un algoritmo para difusión confiable en toda la red.

Bloque 5: Inundación de difusión

Introducción

- En este bloque cambiamos de objetivo: ya no queremos llegar a un destino único, sino **difundir un mensaje a todos los enrutadores de la red**, los algoritmos de caminos más cortos se vuelven conceptualmente inapropiados: están diseñados para optimizar un trayecto punto a punto, no para garantizar cobertura total.
 - Usarlos para difusión implica calcular rutas individuales hacia cada destino, lo cual introduce un costo computacional y de señalización innecesario, además de depender de información de topología que puede no estar disponible o estar desactualizada.
 - La meta aquí es la entrega del paquete a los nodos sin necesidad de precomputar rutas óptimas.

Bloque 5: Inundación de difusión

Introducción

- El problema anterior requiere una variante distinta de inundación.
- Los estudiantes derivarán un mecanismo de control llamado el registro de paquetes difundidos.
- Este bloque funciona como puente conceptual hacia el siguiente, donde la difusión confiable se vuelve un componente esencial del enrutamiento de estado de enlace.

Bloque 5: Inundación de difusión

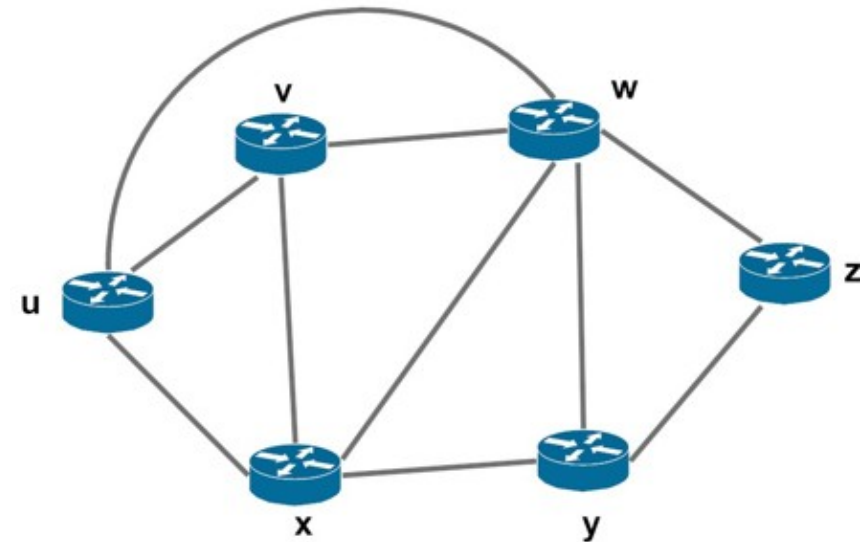
- **Para la situación de difundir un paquete a todos los nodos de una red proponer la idea de un algoritmo**
(dar la idea de inundación gruesa, no pensar en como optimizarla aun)

Bloque 5: Inundación de difusión

- **Para la situación de difundir un paquete a todos los nodos de una red proponer la idea de un algoritmo** (dar la idea de inundación gruesa, no pensar en como optimizarla aun)
- para difundir un paquete desde un origen u se respetan las siguientes reglas:
 - u manda el mensaje por todas las líneas de salida.
 - Cada paquete que llega a un enrutador se reenvía por cada una de las líneas excepto aquella por la que llegó.

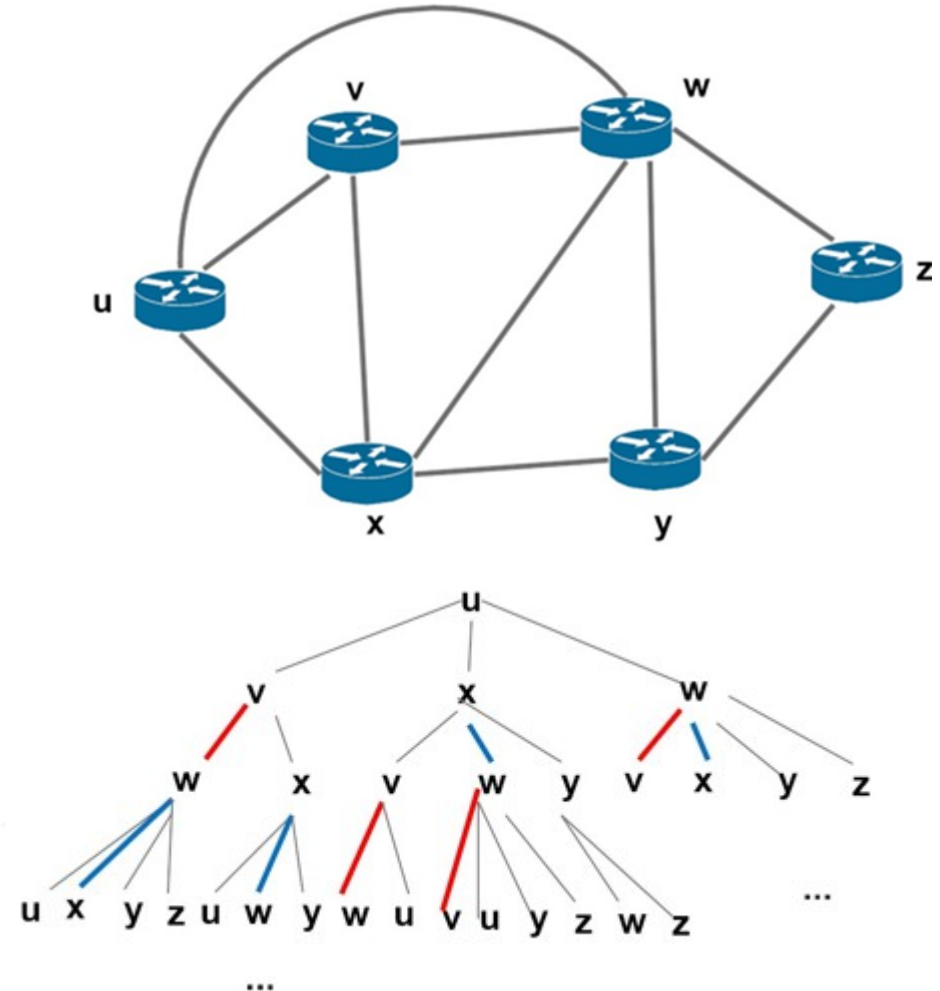
Bloque 5: Inundación de difusión

- **¿Qué problemas tiene la idea anterior?**
(considerar por ejemplo la red de la figura)



Bloque 5: Inundación de difusión

- **Problemas de la idea anterior:**
 - La inundación genera grandes cantidades de *paquetes duplicados*.
 - Árbol de envío de paquetes. Cada arco representa un paquete que se envía.
 - P.ej. arcos rojos y azules en el árbol de arriba corresponden a paquetes duplicados.
 - Árbol de envío de paquetes es infinito con infinitos duplicados. O sea, se generan infinitas rutas.
 - La causa es la presencia de ciclos en el grafo de la subred.



Bloque 5: Inundación de difusión

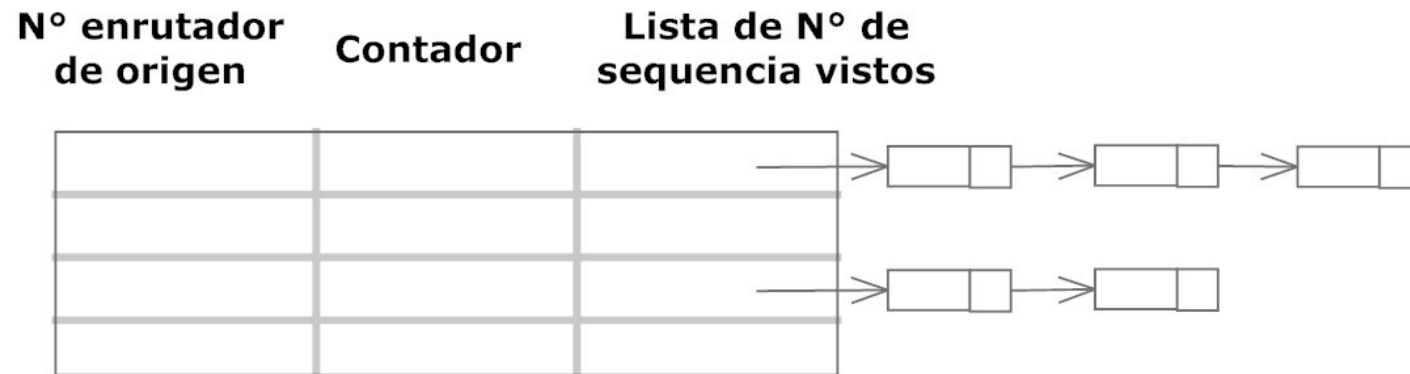
- Hace falta **limitar** un poco el proceso de inundación dado en la idea anterior para resolver el problema.
- **Proponer ideas de solución al problema** (Pensar que cuando llega un paquete a un enrutador, este chequea una condición sobre el paquete para decidir si lo sigue reenviando).

Bloque 5: Inundación de difusión

- Hace falta **limitar** un poco el proceso de inundación dado en la idea anterior para resolver el problema.
- **Proponer ideas de solución al problema** (Pensar que cuando llega un paquete a un enrutador, este chequea una condición sobre el paquete para decidir si lo sigue reenviando).
 - Cada enrutador recuerda los **paquetes difundidos**; por lo tanto si un paquete llega, se chequea si el enrutador lo difundió antes. Si no lo hizo, se lo difunde.
 - El paquete lleva un **contador de cuantos saltos le quedan** que se decrementa con cada salto del paquete. Si el contador es cero, entonces el paquete no se difunde.

Bloque 5: Inundación de difusión

- Vamos a considerar inundación de difusión con registro de paquetes difundidos.
- El registro de paquetes difundidos es la misma estructura de datos que antes.



- **Plantear el algoritmo de inundación de difusión para el caso que llega un paquete a un enrutador.**

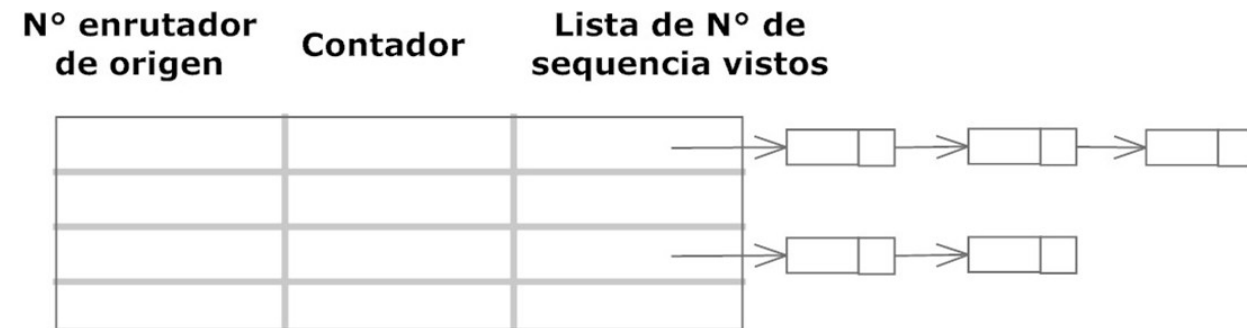
Bloque 5: Inundación de difusión

- **Si el enrutador de origen no está en la tabla:**

1. Se crea una entrada en la tabla para ese enrutador de origen.
2. **Si el número de secuencia es 0:** se agrega 0 en el contador.
3. **Sino** se crea un nodo en la lista de números de secuencia.
4. Se difunde el paquete.

- **Si el enrutador de origen esta en la tabla:**

- **si el número de secuencia es el siguiente al contador:** se actualiza el contador removiendo elementos en la lista de números de secuencia si es necesario. Luego se difunde el paquete.
- **Si el numero de secuencia está en la lista o tiene el valor del contador:** se descarta el paquete.
- **Sino:** se agrega un nodo con el número de secuencia a la lista de números de secuencia. Luego se difunde el paquete.



Bloque 5: Inundación de difusión

- La inundación de difusión nos permitió enviar información a todos los enrutadores, controlando duplicados y evitando ciclos.
 - Ese mecanismo es poderoso, pero todavía no resuelve el problema más general: ¿cómo puede cada enrutador construir una visión consistente de toda la red y actualizarla cuando la topología cambia?
 - Para eso necesitaremos combinar difusión confiable con medición de costos y cálculo de rutas.
 - En el próximo bloque integraremos todas estas ideas para derivar el enrutamiento de estado de enlace.

Bloque 6: Enrutamiento de estado de enlace

- **Problema:** se quiere un algoritmo de enrutamiento que se adapte a cambios en la topología y el tráfico de la red.
- **Idea:** Se usa algoritmo de Dijkstra periódicamente y cada vez que cambia la topología en la red,
 - ❑ En cada una de esas veces se necesita conocer la topología de la subred y los retardos de las líneas.
- Vamos a derivar paso a paso un algoritmo basado en estas ideas.

Bloque 6: Enrutamiento de estado de enlace

Introducción

- En el bloque final integramos todo lo aprendido para abordar el escenario más general y realista:
 - una red donde la topología y el tráfico pueden cambiar dinámicamente, no por hostilidad sino por variaciones normales de operación.
- En este contexto aparece el **enrutamiento de estado de enlace**, un protocolo distribuido que permite a cada enrutador construir una visión consistente de la red y calcular rutas óptimas en función de información actualizada.
- Para derivarlo paso a paso, los estudiantes deberán combinar ideas previas como la inundación de difusión y el algoritmo de Dijkstra.

Bloque 6: Enrutamiento de estado de enlace

Introducción

- Primero construiremos una versión básica del protocolo de estado de enlace y luego analizaremos los problemas técnicos que pueden surgir ante distintos eventos que lo ponen en riesgo.
- Además introduciremos algunas optimizaremos al protocolo que permiten reducir la carga sobre la WAN durante la ejecución del algoritmo de enrutamiento.

Bloque 6: Enrutamiento de estado de enlace

- **¿Qué información puede obtener un enrutador de manera fácil y directa sobre la red donde está?**

Bloque 6: Enrutamiento de estado de enlace

- **¿Qué información puede obtener un enrutador de manera fácil y directa sobre la red donde está?**
 - La información sobre sus vecinos.
 - Para obtenerla puede comunicarse con ellos.
- **¿Qué información se puede obtener de los vecinos?**
(considerar que vamos a derivar un enrutamiento de caminos más cortos)

Bloque 6: Enrutamiento de estado de enlace

- **¿Qué información puede obtener un enrutador de manera fácil y directa sobre la red donde está?**
 - La información sobre sus vecinos.
 - Para obtenerla puede comunicarse con ellos.
- **¿Qué información se puede obtener de los vecinos?**
 - Identificación de los vecinos.
 - Retardos a los vecinos.
- **¿En qué tareas se puede dividir la obtención de información de sus vecinos?**

Bloque 6: Enrutamiento de estado de enlace

- **¿Qué información puede obtener un enrutador de manera fácil y directa sobre la red donde está?**
 - La información sobre sus vecinos.
 - Para obtenerla puede comunicarse con ellos.
- **¿Qué información se puede obtener de los vecinos?**
 - Identificación de los vecinos.
 - Retardos a los vecinos.
- **¿En qué tareas se puede dividir la obtención de información de sus vecinos?**
 - 1.Descubrir sus vecinos
 - 2.Medir el costo a cada uno de sus vecinos

Bloque 6: Enrutamiento de estado de enlace

- Supongamos que cada enrutador cuenta con la información de los vecinos y retardos a ellos. **¿Cómo se puede hacer para que cada enrutador conozca la topología entera de la red?**

Bloque 6: Enrutamiento de estado de enlace

- Supongamos que cada enrutador cuenta con la información de los vecinos y retardos a ellos. **¿Cómo se puede hacer para que cada enrutador conozca la topología entera de la red?**
- La información de retardos a sus vecinos que obtuvo cada enrutador se distribuye a todos los demás enrutadores de la red.
- **¿Cómo se puede distribuir por toda la red la información de los retardos a sus vecinos que tiene un enrutador?**

Bloque 6: Enrutamiento de estado de enlace

- Supongamos que cada enrutador cuenta con la información de los vecinos y retardos a ellos. **¿Cómo se puede hacer para que cada enrutador conozca la topología entera de la red?**
- La información de retardos a sus vecinos que obtuvo cada enrutador se distribuye a todos los demás enrutadores de la red.
- **¿Cómo se puede distribuir por toda la red la información de los retardos a sus vecinos que tiene un enrutador?**
- Usar inundación de difusión.
- **¿En qué tareas se puede dividir la distribución por la red de la información de los retardos a sus vecinos que tiene un enrutador?**

Bloque 6: Enrutamiento de estado de enlace

- Supongamos que cada enrutador cuenta con la información de los vecinos y retardos a ellos. **¿Cómo se puede hacer para que cada enrutador conozca la topología entera de la red?**
- La información de retardos a sus vecinos que obtuvo cada enrutador se distribuye a todos los demás enrutadores de la red.
- **¿Cómo se puede distribuir por toda la red la información de los retardos a sus vecinos que tiene un enrutador?**
- Usar inundación de difusión.
- **¿En qué tareas se puede dividir la distribución por la red de la información de los retardos a sus vecinos que tiene un enrutador?**
 1. El enrutador construye un paquete del estado de los enlaces a sus vecinos.
 2. Se envía ese paquete a todos los enrutadores usando inundación de difusión.

Bloque 6: Enrutamiento de estado de enlace

- Juntando todo: **¿Qué tareas hace un enrutador?**

Bloque 6: Enrutamiento de estado de enlace

- Juntando todo: **¿Qué tareas hace un enrutador?**
 1. Descubrir sus vecinos
 2. Medir el costo a cada uno de sus vecinos
 3. Construir un paquete diciendo lo que ha aprendido
 4. Enviar este paquete a todos los demás enrutadores
 5. Computar el camino más corto a cada uno de los otros enrutadores
- Este algoritmo es valioso porque:
 - **Responde rápido** frente a cambios en la topología de la red.
 - Ayuda a derivar algoritmos de enrutamiento más sofisticados como el que usa internet (OSPF)
- Ahora vamos a ver cada una de las tareas anteriores en detalle.

Bloque 6: Enrutamiento de estado de enlace

- **¿Cómo se puede averiguar quiénes son los vecinos de un enrutador?** (ese enrutador puede mandar paquetes a ellos)

Bloque 6: Enrutamiento de estado de enlace

- ¿Cómo se puede averiguar quiénes son los vecinos de un enrutador?
 - Se envía **paquete Hello** a cada línea punto a punto
 - Se espera que el enrutador del otro extremo regrese una respuesta indicando quién es.

Bloque 6: Enrutamiento de estado de enlace

- ¿Cómo se puede hacer para que enrutador conozca retardo a sus vecinos? (ya sabe quienes son sus vecinos)

Bloque 6: Enrutamiento de estado de enlace

- **¿Cómo se puede hacer para que enrutador conozca retardo a sus vecinos?** (ya sabe quienes son sus vecinos)
 - enviar un **paquete ECHO** especial a través de la línea
 - Una vez que llegue al otro extremo, éste debe regresarlo inmediatamente
 - Uso de temporizadores para medir el tiempo.
 - **Método:** Se mide el tiempo de ida y vuelta y se divide por 2.
- **Problema del método:**
 - Asume implícitamente que los retardos son simétricos

Bloque 6: Enrutamiento de estado de enlace

- Cada enrutador construye un **paquete de estado de enlace** (LSP) conteniendo los retardos a sus vecinos.
- **¿Qué datos debe tener un LSP?**

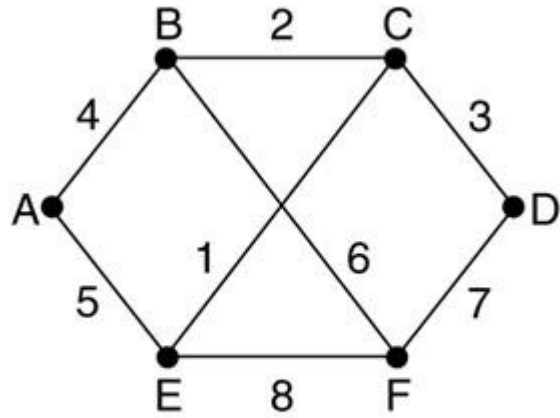
Bloque 6: Enrutamiento de estado de enlace

- Cada enrutador construye un **paquete de estado de enlace** (LSP) conteniendo los retardos a sus vecinos.
- **¿Qué datos debe tener un LSP?**
 - Identidad del emisor (para saber de quien se trata)
 - Número de secuencia (para distinguir entre distintos LSP de un enrutador)
 - Lista de <vecino, retardo al vecino>
- **¿Cuándo se pueden construir los LSP?**

Bloque 6: Enrutamiento de estado de enlace

- Cada enrutador construye un **paquete de estado de enlace** (LSP) conteniendo los retardos a sus vecinos.
- **¿Qué datos debe tener un LSP?**
 - Identidad del emisor (para saber de quien se trata)
 - Número de secuencia (para distinguir entre distintos LSP de un enrutador)
 - Lista de <vecino, retardo al vecino>
- **¿Cuándo se pueden construir los LSP?**
 - Construirlos a intervalos regulares.
 - Construirlos cuando ocurra un evento significativo, como la caída o la reactivación de la línea o de un vecino, o el cambio apreciable de sus propiedades.

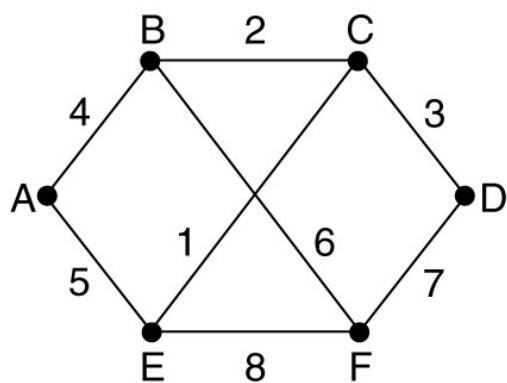
Bloque 6: Enrutamiento de estado de enlace



(a)

¿Qué LSPs se construyen para
nodos A y B?

Bloque 6: Enrutamiento de estado de enlace



(a)

Subred

		Link		State		Packets	
A		B		C		D	
Seq.		Seq.		Seq.		Seq.	
Age		Age		Age		Age	
B	4	A	4	B	2	C	3
E	5	C	2	D	3	F	7
		F	6	E	1		
						A	5
						C	1
						F	8
						B	6
						D	7
						E	8

(b)

LSP para cada nodo

Bloque 6: Enrutamiento de estado de enlace

- Ahora vemos la distribución confiable de los LSP.
- Usamos inundación de difusión con registro de paquetes difundidos.
- **¿Cómo es la estructura de datos del registro de paquetes difundidos que lleva cada enrutador?** (Tener en cuenta que no hace falta guardar varios números de secuencia de LSP que se quedaron viejos)

Bloque 6: Enrutamiento de estado de enlace

- Ahora vemos la distribución confiable de los LSP.
- Usamos inundación de difusión con registro de paquetes difundidos.
- **¿Cómo es la estructura de datos del registro de paquetes difundidos que lleva cada enrutador?** (Tener en cuenta que no hace falta guardar varios números de secuencia de LSP que se quedaron viejos)
- Basta con para un enrutador de origen indicar el último número de secuencia ya visto de ese enrutador de origen (este sería el LSP más reciente recibido).
- **Describir el algoritmo de inundación de difusión usando una tabla de pares <enrutador de origen, último número de secuencia>**

Bloque 6: Enrutamiento de estado de enlace

- Cuando llega un LSP a un enrutador,
 - Si es nuevo (nuevo número de secuencia mayor que los anteriores),
 - se reenvía a través de todas las líneas, excepto aquella por la que llegó.
 - Si es un duplicado (número de secuencia mayor visto, pero repetido),
 - se descarta.
 - Si llega un paquete con número de secuencia menor que el mayor visto hasta el momento,
 - se rechaza como **obsoleto** debido a que el enrutador tiene datos más recientes.

Bloque 6: Enrutamiento de estado de enlace

- ¿Para qué son los LSP que llegan a un enrutador?

Bloque 6: Enrutamiento de estado de enlace

- ¿Para qué son los LSP que llegan a un enrutador?
- Para construir el grafo de la red de enrutadores interconectados.
- Por lo tanto es necesario almacenar en búfer en el enrutador los LSP más recientes recibidos de cada origen.
- ¿Qué elementos puede contener una fila de la tabla del búfer de LSP de un enrutador?
- Enrutador de origen, número de secuencia del último LSP, datos de ese LSP.

Source	Seq.	Data
A	21	
F	21	
E	21	
C	20	
D	21	

Bloque 6: Enrutamiento de estado de enlace

- ¿Cuándo se puede crear o actualizar la tabla de enrutamiento de un enrutador?

Bloque 6: Enrutamiento de estado de enlace

- ¿Cuándo se puede crear o actualizar la tabla de enrutamiento de un enrutador?
- una vez que el enrutador ha acumulado un grupo completo de paquetes de estado del enlace
- ¿Qué se hace después?

Bloque 6: Enrutamiento de estado de enlace

- ¿Cuándo se puede crear o actualizar la tabla de enrutamiento de un enrutador?
 - una vez que el enrutador ha acumulado un grupo completo de paquetes de estado del enlace
 - ¿Qué se hace después?
1. Usando los LSP construir el grafo de la subred completa.
 - ☐ Cada enlace se representa dos veces, una para cada dirección.
 - ☐ Los dos valores pueden promediarse o usarse por separado.
 2. Se ejecuta el **algoritmo de Dijkstra** para construir la ruta más corta a todos los destinos posibles.
 3. Con los resultados del mismo se actualiza la tabla de enrutamiento.

Bloque 6: Enrutamiento de estado de enlace

De la versión básica a un protocolo robusto y eficiente

- Ya derivamos una primera versión del protocolo de estado de enlace: descubrimos vecinos, medimos costos, construimos LSPs, los difundimos y ejecutamos Dijkstra.
- Pero un protocolo real enfrenta problemas adicionales: por ejemplo, errores en números de secuencia y caída de enrutadores. Además es necesario de reducir la carga sobre la red.
- En la siguiente parte del bloque veremos cómo resolver estos problemas técnicos y cómo optimizar el protocolo para hacerlo más eficiente y más robusto.

Bloque 6: Enrutamiento de estado de enlace

- **Situación:** *Si llega a corromperse un número de secuencia y se escribe 65540 en lugar de 4 (un error de un bit), los paquetes 5 a 65540 serán rechazados como obsoletos, dado que se piensa que el número de secuencia actual es 65540.*
- Para protegerse contra errores en las líneas entre enrutadores se puede confirmar cada LSP que se recibe. **¿Cómo funciona entonces la protección contra estos errores?** (sugerir procedimiento)

Bloque 6: Enrutamiento de estado de enlace

- **Situación:** *Si llega a corromperse un número de secuencia y se escribe 65540 en lugar de 4 (un error de un bit), los paquetes 5 a 65540 serán rechazados como obsoletos, dado que se piensa que el número de secuencia actual es 65540.*
- Para protegerse contra errores en las líneas entre enrutadores se puede confirmar cada LSP que se recibe. **¿Cómo funciona entonces la protección contra estos errores?**
 - Antes de actualizarse el número de secuencia más grande, el enrutador manda una confirmación de recepción al transmisor y luego espera una respuesta afirmativa o negativa del transmisor.
 - En el primer caso se actualiza el número de secuencia más grande.
 - En el segundo caso se descarta el LSP que se recibió por estar errado.

Bloque 6: Enrutamiento de estado de enlace

- Asumir que una vez que un LSP más actualizado de un enrutador de origen llega a un enrutador, y no se lo encola para difusión inmediata, sino que se espera un tiempo breve. **¿Cómo esto cambia la manera a hacer la difusión?** (analizar qué puede pasar durante esa espera)

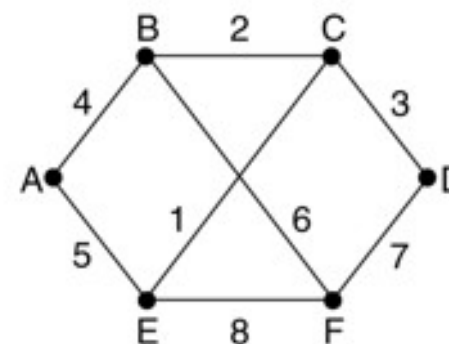
Bloque 6: Enrutamiento de estado de enlace

- Asumir que una vez que un LSP más actualizado de un enrutador de origen llega a un enrutador, y no se lo encola para difusión inmediata, sino que se espera un tiempo breve. **¿Cómo esto cambia la manera a hacer la difusión?**
 - En ese tiempo puede llegar desde otras líneas el mismo LSP, por lo que no va a ser necesario difundir el LSP por esas líneas.
 - También puede llegar un LSP más reciente del mismo origen, por lo que se cambia el LSP en el buffer por otro más reciente evitando así enviar un LSP que se quedó viejo.
 - Por lo tanto se puede **hacer más eficiente** el algoritmo de inundación.
- **¿Cómo modificar el búfer de LSPs para reflejar esta optimización?**
(recordar que además de difundir el LSP hace falta confirmarlo)

Bloque 6: Enrutamiento de estado de enlace

- Usar **Banderas** que pueden ser:
 - **Banderas de confirmación de recepción**: indica a dónde tiene que enviarse la confirmación de recepción del paquete.
 - **Banderas de envío**: significan que el paquete debe enviarse a través de las líneas indicadas.
 - Si llega un duplicado mientras el original aún está en el búfer, los bits de las banderas tienen que cambiar.

Source	Seq.	Age	Send flags			ACK flags			Data
			A	C	F	A	C	F	
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	



Búfer para el enrutador B

Bloque 6: Enrutamiento de estado de enlace

- **Problema:** Si los números de secuencia vuelven a comenzar, reinará la confusión (el algoritmo de inundación no está preparado para eso).
- **¿Cómo evitar este problema?**

Bloque 6: Enrutamiento de estado de enlace

- **Problema:** Si los números de secuencia vuelven a comenzar, reinará la confusión (el algoritmo de inundación no está preparado para eso).
- **¿Cómo evitar este problema?**
- Usar un número de secuencia de longitud suficiente para que el problema anterior no suceda. Por ej. de 32 bits.
 - ❑ P.ej. Si un enrutador produce un paquete de estado de enlace cada segundo, llevará 137 años antes de volver a empezar.

Bloque 6: Enrutamiento de estado de enlace

- **Problema:** *Si llega a caerse un enrutador (de origen), perderá el registro de su número de secuencia. Si comienza nuevamente en 0, se rechazará el siguiente paquete.*
- Para evitar que esto suceda se puede hacer que la información de un enrutador caído expire a lo largo de la red luego de caerse.
- **Sugerir una implementación de esta idea.**

Bloque 6: Enrutamiento de estado de enlace

- **Problema:** *Si llega a caerse un enrutador (de origen), perderá el registro de su número de secuencia. Si comienza nuevamente en 0, se rechazará el siguiente paquete.*
- Para evitar que esto suceda se puede hacer que la información de un enrutador caído expire a lo largo de la red luego de caerse.
- **Sugerir una implementación de esta idea.**
 - ❑ **Una vez identificado que un enrutador está caído:**
 - Se propaga la información de este hecho por toda la red.
 - Se hace que la información asociada al enrutador caído expire (paquete pendiente a enviar, número de secuencia más grande recibido, etc.).
 - Así que cuando ese enrutador vuelva a la vida, puede comenzar con número de secuencia 0.

Bloque 6: Enrutamiento de estado de enlace

- **Recapitulación:** En este bloque integramos todo lo aprendido para derivar un protocolo capaz de adaptarse a cambios normales en la topología y el tráfico de la red.
 - Construimos paso a paso el enrutamiento de estado de enlace, combinando descubrimiento de vecinos, medición de costos, construcción de LSPs, inundación de difusión confiable y ejecución de Dijkstra sobre la base de datos de estado de enlace.
 - También analizamos problemas técnicos que pueden surgir (errores en números de secuencia, que los números de secuencia den vuelta, y caída de enrutadores) y vimos cómo resolverlos y optimizar el protocolo para reducir el tráfico en la WAN.